Contents

1 Introduction **6**

1.1 Background 6

1.2 Research Gap 7

1.3 Approach 7

1.4 Contributions 8

1.5 Paper Outline 9

2 Literature Review **10**

2.1 Language Server Protocol 10

2.2 Rzk 11

2.3 Typechecker as a Build System 11

2.4 Document processing in various ITPs 12

2.4.1 Roqc 12

2.4.2 Summary 13

3 Methodology **14**

4 Implementation **15**

5 Evaluation and Discussion **16**

6 Conclusion 17

Bibliography cited 18

A Extra Stuff 20

B Even More Extra Stuff 21

List of Tables

1.1 Evaluted diagnostic latency of Rzk, its current language server and its implementation proposed in this thesis on the sHoTT project (computation time in ms) 9

2.1 Caching and evaluation triggers in ITPs 13

List of Figures

Abstract

Rzk is an Interactive Theorem Prover (ITP) designed for formalizing modern mathematics, featuring a built-in language server. However, its existing feedback loop is slow and relies on explicit user actions. This paper presents the design, implementation, and evaluation of an enhanced LSP server for Rzk. The proposed architecture introduces advancements to type-checking and caching mechanisms, enabling the typechecker to read unsaved IDE buffers, compute diagnostics incrementally, and update them dynamically in response to user input.

Chapter 1

Introduction

1.1 Background

Interactive Theorem Provers[1] are domain-specific languages designed for formalizing mathematics. They are essential for modern mathematics research, granting a near-absolute degree of certainty by verifying proofs via deterministic algorithms and managing extreme complexity by decomposing large-scale proofs into reusable components. Therefore, they are frequently utilized by mathematicians who may lack a comprohensive programming background to employ ITPs effectively.

Integrated Development Environment[2] integration with ITP represents a viable approach to addressing this challenge by equipping user with rapid, real-time code diagnostics, seamless code navigation, syntax highlighting, and other advanced features.

[1] Commonly abbreviated as ITPs

[2] Commonly abbreviated as IDEs

1.2 Research Gap

Rzk is a theorem prover based on Riehl and Shulman's type theory[1] with shapes. While its existing language server keeps track of unsaved changes, highlights syntax and enables navigation, it suffers from several limitations: proof evaluation is not automatically executed upon file modification, and large projects exhibit significant latency during evaluation when changes are introduced at the beginning of the codebase.

1.3 Approach

To achieve these objectives, the research presented in this thesis is structured in several consecutive stages.

Initially, the typechecker module of language server will be adapted to leverage unsaved changes from the IDE where possible, thereby providing relevant diagnostics.

Subsequently, an edit-triggered evaluation policy will be introduced to deliver these diagnostics automatically without requiring explicit manual triggers.

These enhancements establish the foundation for the core contribution of this thesis, enabling the evaluation of diagnostic delivery latencies within the current architecture of the Rzk LSP[3] server.

Following this, a build system incorporating module dependencies and cache invalidation will be designed and implemented to eliminate redundant type checking of unmodified code, thereby minimizing the latency of diagnostic delivery.

[3]Language Server Protocol

1.4 Contributions

In summary, this thesis delivers several contributions that optimize IDE behavior as follows:

- The language server provides relevant diagnostics for unsaved changes directly within the IDE. In this context, "relevant diagnostics" implies that diagnostics for unsaved changes matches the output of the Rzk typechecker executed on the corresponding source files on disk.

- When user begins typing, outdated diagnostic messages are immediately suppresed. Following a 500 ms timeout after the final keystroke, the language server triggers typecheck on the project, presenting results as fresh dianostics. This debouncing strategy allows language server to initiate computations promptly while mitigating the overhead of repeatedly halting and restarting the checker during rapid continuous typing.

- The language server minimizes computational overhead by avoiding repetitive execution on unchanged code. When a user modifies a module, typecheckind is isolated to that specific module and its downstream dependencies, rather than every subsequent file in the project hierarchy. Although this optimization is less noticeable to the user, this strategy is expected to substantially reduce re-evaluation latencies for large projects when early-stage files are modified, while maintaining standard performance for changes made in later files. For instance, the evaluated performance metrics of Rzk and its language server on the simplicial HoTT formalization project (sHoTT)[2] - which comprises 13 files with rzk code - are present in Table 1.1.

1.5 Paper Outline

The remainder of this paper is structured as follows. Chapter 2 reviews Rzk, the LSP, and comparable solutions implemented in other ITPs. Chapter 3 describes the applied methodology. Chapter 4 documents the design and implementation aspects. Chapter 5 presents and analyzes the experimental results. Chapter 6 concludes the paper and summarizes the completed work.

	Whole project	00-common.rzk.md	13-truncation-levels
Rzk	3000	200	300
Current language server	3000	3000	300
Improved language server	3000	200	300

TABLE 1.1
Evaluted diagnostic latency of Rzk, its current language server and its implementation proposed in this thesis on the sHoTT project (computation time in ms)

Chapter 2

Literature Review

2.1 Language Server Protocol

The Language Server Protocol developed by Microsoft[3], has profoundly influenced the development of both domain-specific languages[1] and integrated development environments by decoupling them via a standardized set of requests. Consequently, DSL developers no longer need to implement and maintain language-specific logic for each supported IDE; conversely, IDE integration requires significantly less effort by leveraging these standardized API calls.

After studying the impact of LSP in [4], Hendrik Büender concludes that the introduction of the protocol not only substantially reduced development effort, but also defined "minimum feature set that should be supported by any DSL editor" and even enabled the complete omission of local IDE installation in certain scenarios.

[1] Commonly abbreviated as DSLs

2.2 Rzk

Rzk is the interactive theorem prover developed "with the idea of bringing Riehl and Shulman's paper[1] to life by implementing a proof assistant based on their type thory with shapes", as stated in its official GitHub repository[5].

It has already been utilized in several mathematical formalization projects:

- Formalizing the ∞-Categorical Yoneda Lemma [6]
- Simplicial HoTT and synthetic ∞-categories [2]

2.3 Typechecker as a Build System

In [7], Mokhov et al. propose a systematic framework for developing build systems. This framework conceptualizes a build system as two architectural components: a "Scheduler", handles task scheduling in dependency order, and a "Rebuilder", which determines whether a task requires re-execution.

The Rzk language server compiles a project into a collection of diagnostics (errors and warnings), storing an intermediate representation that contains declarations and diagnostics for each module. By treating project files as "inputs", module caches as "outputs", and the typechecking process itself as the primary task, the aforementioned framework can be leveraged to develop an incremental build system. This approach eliminates repetitive checking of unmodified code, thereby lowering latency under specific conditions.

Other notable frameworks exist within the domain of incremental computation systems:

- In [8], Hammer et al. build upon the concept of "demanded computations

graph" to present theoretical "core calculus that applies a demand-driven semantics to incremental computation", implementing this calculus as an OCaml library named Adapton.

- The rust-analyzer team developed Salsa[9], a Rust framework that utilizes a similar incremental, demand-driven computation.

Despite both framework introducing powerful paradigms for implementing incremental computation, they are fundamentally based on the demand-driven paradigm[10]. This contrasts with Rzk, which enforces a strict project structure and requires full compilation. Furthermore, it is noteworthy that both Rzk and framework presented in [7] are implemented in Haskell, whereas Adapton is an OCaml library and Salsa is written in Rust.

In conclusion, the build system framework proposed by Mokhov et al. is more aligned with the objectives of this thesis, though specific architectural details from Adapton and Salsa may still influence the final implementation.

2.4 Document processing in various ITPs

2.4.1 Roqc

Roqc Prover (formerly known as Coq) is an interactive theorem prover based on the Calculus of Inductive Constructions.

Prior to the introduction of the LSP, Roqc utilized a proprietary implementation of asynchronous document processing, as described in [11]. That work demonstrates significant latency reduction both during the batch compilation of entire project and during on-edit evaluation within CoqIDE. However, that ap-

proach focuses on parallel and asyncronous processing of independent documents rather than on incremental computation of unified project.

As noted in its package documentation[12], the Rocq LSP server (coq-lsp) compiles code incrementally and continuously validates documents as user edits arrive.

Rocq compiles its '.v' source code files into '.vo' object files. Because the feedback loop in IDE is based on importing these pre-compiled object files, the process inherently exhibits incremental characteristics.

2.4.2 Summary

To conclusde this section, the capabilities and behaviors of these varius environments are summarized in Table2.1.

Language server	Reevaluation trigger	Project-level cache	Cache structure
Rocq	Edit	Compiled modules	Module imports

TABLE 2.1
Caching and evaluation triggers in ITPs

Chapter 3

Methodology

...

Chapter 4

Implementation

...

Chapter 5

Evaluation and Discussion

...

Chapter 6

Conclusion

...

Bibliography cited

[1] E. Riehl and M. Shulman, *A type theory for synthetic ∞-categories*, 2023. arXiv: 1705.07442 [math.CT]. [Online]. Available: https://arxiv.org/abs/1705.07442

[2] N. Kudasov, "Simplicial hott and synthetic ∞-categories (codebase)," [Online]. Available: https://github.com/rzk-lang/sHoTT

[3] *Language Server Protocol Specification*, 2026. [Online]. Available: https://microsoft.github.io/language-server-protocol/specifications/lsp/3.18/specification/

[4] H. Büender, "Decoupling language and editor - the impact of the language server protocol on textual domain-specific languages," Jan. 2019, pp. 129–140. DOI: 10.5220/0007556301290140 [Online]. Available: https://pdfs.semanticscholar.org/8526/1fa85335a396934ddf20277fb85fc3bd309d.pdf

[5] N. Kudasov, "Rzk proof assistant github repository," 2026. [Online]. Available: https://github.com/rzk-lang/rzk

[6] N. Kudasov, E. Riehl, and J. Weinberger, "Formalizing the ∞-categorical yoneda lemma," in *Proceedings of the 13th ACM SIGPLAN International Conference on Certified Programs and Proofs*, ser. CPP 2024, London, UK: Association for Computing Machinery, 2024, pp. 274–290, ISBN:

9798400704888. DOI: 10.1145/3636501.3636945 [Online]. Available: https://doi.org/10.1145/3636501.3636945

[7] A. Mokhov, N. Mitchell, and S. Peyton Jones, "Build systems à la carte," *Proc. ACM Program. Lang.*, vol. 2, no. ICFP, Jul. 2018. DOI: 10.1145/3236774 [Online]. Available: https://doi.org/10.1145/3236774

[8] M. A. Hammer, K. Y. Phang, M. Hicks, and J. S. Foster, "Adapton: Composable, demand-driven incremental computation," *SIGPLAN Not.*, vol. 49, no. 6, pp. 156–166, Jun. 2014, ISSN: 0362-1340. DOI: 10.1145/2666356.2594324 [Online]. Available: https://doi.org/10.1145/2666356.2594324

[9] *Salsa*. [Online]. Available: https://salsa-rs.github.io/salsa/

[10] L. Duràn, "Incremental computations with salsa and adapton frameworks," Tech. Rep., Oct. 2025. [Online]. Available: https://lucioduran.com/blog/incremental-computation-salsa-adapton-frameworks

[11] B. Barras, C. Tankink, and E. Tassi, *Asynchronous processing of coq documents: From the kernel up to the user interface*, 2015. arXiv: 1506.05605 [cs.LO]. [Online]. Available: https://arxiv.org/abs/1506.05605

[12] *Coq-lsp*. [Online]. Available: https://ocaml.org/p/coq-lsp/0.1.2%2Bv8.16/doc/README.html

Appendix A

Extra Stuff

...

Appendix B

Even More Extra Stuff

...